

Q1: Count number of instances of a class created in Python?

Example: Say `Car` is any class.

```
maruti = Car()
bmw = Car()
honda = Car()
```

So after creating above instances. We want to count how many instances are created of `Car` class.

For above example `no of instances = 3`.

Write a program for above problem.

In []:

```
#write your code here
```

Q-2: Create a deck of cards class. Internally, the deck of cards should use another class, a card class. Your requirements are:

- The `Deck` class should have a `deal` method to deal a single card from the deck
- After a card is dealt, it is removed from the deck.
- There should be a `shuffle` method which makes sure the deck of cards has all 52 cards and then rearranges them randomly.
- The `Card` class should have a `suit` (Hearts, Diamonds, Clubs, Spades) and a `value` (A,2,3,4,5,6,7,8,9,10,J,Q,K)

Deck Class

- It is class of all possible cards in a deck. Total 52 cards.
- **Methods** - `deal()` it will take out one card from the deck of cards.
- Deck of cards should get shuffled while creating the deck object.
- `no of cards remaining in deck - <number>` should display on printing any deck object.

Card class

- It is a class of card
- **Attributes** - `suit` and `value`
- `<suit> of <value>` should display on printing any card object.

In []:

```
#Code Here
```

Q-3: Find the area of a rectangle.

Approach:

- The class name should be *Rectangle*.
- The constructor should accept two parameters *length* and *height* but you can't pass the values directly to it while creating the constructor. E.g., `rectangle = Rectangle(length=10, height=8)` <-- you can't do that while creating the instances.
- Create a method called *area()* which has no parameters.
- Create a method called *is_square()* which also has no parameters. Return `True` if the rectangle is a square otherwise return `False`.
- If you are using a if-else block inside the *is_square()* method, then use the one-linear syntax.

In []:

```
#Write your code here
```

Q-4 : Problem 4

Statement: Write a program that uses datetime module within a class. Enter manufacturing date and expiry date of the product. The program must display the years, months and days that are left for expiry.

In []:

```
#Write your code here
```

Q-5: Problem 5

Statement: A university wants to automate their admission process. Students are admitted based on the marks scored in the qualifying exam. A student is identified by student id, age and marks in qualifying exam. Data are valid, if:

- Age is greater than 20
- Marks is between 0 and 100 (both inclusive)

A student qualifies for admission, if

- Age and marks are valid and
- Marks is 65 or more

Write a python program to represent the students seeking admission in the university. The details of student class are given below.

Class name: Student

Attributes (private)	student_id	
	marks	
	age	
Methods (public)	__init__()	Create and initialize all instance variables to None
	validate_marks()	If data is valid, return true. Else, return false
	validate_age()	
	check_qualification())	Validate marks and age. • If valid, check if marks is 65 or more. ◦ If so return true ◦ Else return false • Else return false
	setter methods	Include setter methods for all instance variables to set its values

	getter methods	Include getter methods for all instance variables to get its values
--	----------------	---

In []:

#Code here

Q-6: Ice-Cream Scoops and Bowl shop

1. Create a class `Scoop` which has one public property `flavor` and one private property `price`. Take `flavor` values during object creation.
2. Create a class `Bowl` with private property `scoop_list` which will have list of `scoop` object.
3. Create a method `add_scoops` in `Bowl` class which will add any no of `Scoop` objects given as parameter and store it in `scoops_list`.
4. Make getter and setter method for `price` property.
5. Make a method `display` to display `flavour` and `price` of each `Scoop` in `scoop_list` and print total price of the bowl by adding all flavour scoops prices.
6. Make a method `sold` in both `Scoop` class and `Bowl` class to print no of quantity sold.

Ex.-

```
choco = Scoop('chocolate')
print(choco)
choco.set_price(100)

berry = Scoop('berry')
berry.set_price(120)
print(berry)

vanilla = Scoop('vanilla')
vanilla.set_price(150)

bowl = Bowl()

bowl.add_scoops(choco) # Giving one parameter
bowl.add_scoops(berry, vanilla) # Multiple
# add_scoops should handle both scenerios

print(bowl)

bowl.display()

Scoop.sold()
Bowl.sold()
```

Output

```
Flavor - chocolate Price - None
Flavor - berry Price - 120
chocolate
berry
vanilla
Dsisplaying Bowl
Flavor - chocolate Price - 100
Flavor - berry Price - 120
```

```
Flavor - vanilla Price - 150
Price of Bowl - 370
3 scoops sold
1 bowls sold
```

In []:

```
#Write your code here
```

Q-7: Ice-Cream Bowl continue..

Making advancement in the above classes. `Scoop` and `Bowl`

1. Introduce a property `max_scoops` in `Bowl` class to signify maximum scoops that a bowl can have, exceeding that it will display `Bowl is full` . Take default value as `3` .
2. `no_of_scoop` in `Scoop` class with default value of `1`
3. Print `<flavour>` added with every scoop added.

```
#Testing Code-1
```

```
choco = Scoop('chocolate', 1)
choco.set_price(100)
print(choco)
```

```
berry = Scoop('berry', 2)
berry.set_price(120)
print(berry)
```

```
vanilla = Scoop('vanilla') # no of scoop parameter not given, will take default va
lue
vanilla.set_price(150)
print(vanilla)
```

Output:

```
Flavor - chocolate, No of Scoops - 1, Price - 100
Flavor - berry, No of Scoops - 2, Price - 120
Flavor - vanilla, No of Scoops - 1, Price - 150
```

Testing code-2 considering above test code executed

```
bowll = Bowl() # max_scoop parameter not given, will take default value
bowll.add_scoops(choco) # Giving one parameter
bowll.add_scoops(berry, vanilla) # Multiple
bowll.display()
```

Output

```
chocolate added
berry added
Bowl is full
Dsisplaying Bowl-1
Flavor - chocolate, No of Scoops - 1, Price - 100
Flavor - berry, No of Scoops - 2, Price - 120
Price of Bowl-1: 340
```

Testing code-3, considering above tests executed

```
#Tetsing code-3
```

```
bowl2 = Bowl(2)
bowl2.add_scoops(berry)
bowl2.add_scoops(choco)
```

```
bowl2.display()
Bowl.sold()
```

Output:

```
berry added
chocolate not added. Bowl is full!!
Dsiplaying Bowl-2
Flavor - berry, No of Scoops - 2, Price - 120
Price of Bowl-2: 240
2 bowls sold
```

In []:

```
#write your code here
```

In []:

In []:

In []: